



Fundamentals of Version Control & Git

From idea to structured timeline – an introduction to version control, Git installation, and the first steps with the most powerful tool in software development.

CORE TECH FUNDAMENTALS

What is Version Control?

The problem without VCS

Files like `project_final`, `project_final_new`, `project_final_really_final`
– chaotic, error-prone, and impossible to track.

The solution: A time log

A version control system manages changes **cleanly, traceably, and structurally**. It answers: **what** was changed, **when**, **by whom**
– and often **why**, if meaningful commit messages were written.

According to the official Git documentation, changes should remain historically traceable, previous states should be recoverable, and collaboration should be possible in a controlled manner.

Principle 1: Traceable Project History

The heart of a version control system is its **history**. In Git, a saved state is called a **commit**. A commit contains:

File State

State of all tracked files at the time of the commit

Metadata

Author, email address, and timestamp

Commit Message

Description of the changes made

Reference

Pointer to previous commits – forming a chain



Principle 2: Snapshots instead of file differences

How Git stores data

Git treats data conceptually as **snapshots**. When committing, Git does not just remember "line 7 was changed", but saves the state of all tracked files like a **snapshot of the entire project**.

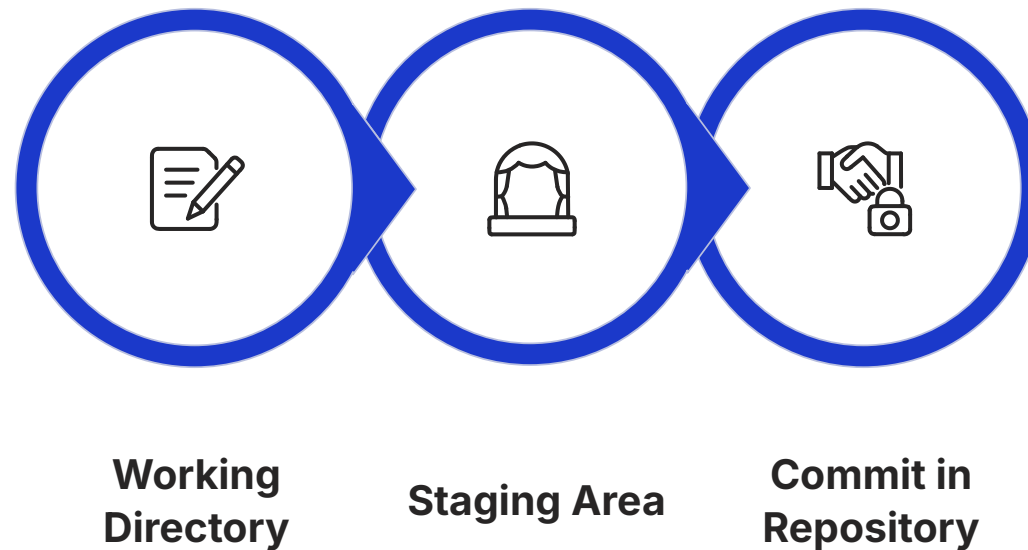
The Git documentation explicitly states that Git thinks more in terms of snapshots than in classic line-based deltas.

Why this is important

Many beginners believe that Git only saves "lists of changes". The internal model, however, is closer to a set of **momentary recordings**. This helps with understanding branches, merges, and restoration.

Principle 3: Conscious Committing of Changes

Git does not automatically save every intermediate movement. **You** decide when a state is important enough.



This conscious committing forces you to structure work steps meaningfully. Good commits make projects more understandable and collaboration more efficient.

Principles 4–6: Comparing, Branching, Integrity

Comparing Changes

Version control is also a **comparison tool**. When an error occurs, you can specifically look at when it originated and which change caused it.

Parallel Work with Branches

Git manages parallel development lines (**branches**). Changes should be testable in isolation and integrated later – without endangering the stable main state.

Integrity & Traceability

Contents are internally identified with **checksums**. Git uniquely recognizes changes; manipulations or corruption are more easily detected. The Git introduction explicitly emphasizes this integrity.

VERSION CONTROL MODELS

Centralized Version Control

In **centralized version control** (e.g., SVN, CVS), the official project history resides on a **single central server**. Developers work with working copies and communicate constantly with this server.

Advantages

- Clear "single source of truth"
- Simple management & permission control
- Conceptually easy to understand

Disadvantages

- Server failure = critical bottleneck
- Many operations are server-dependent
- Local copy lacks complete history
- Less flexible for modern workflows



VERSION CONTROL MODELS

Distributed Version Control

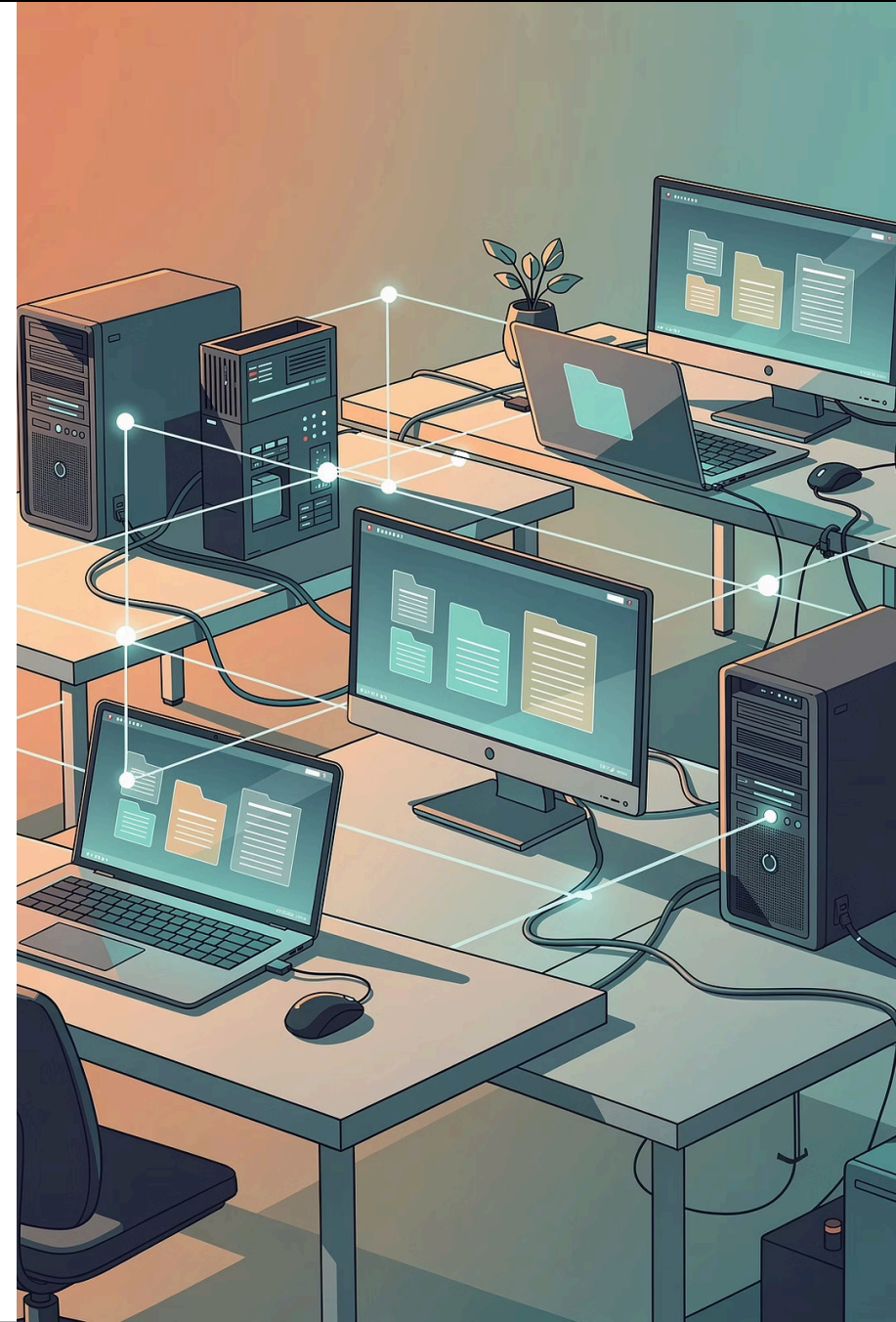
With **distributed version control**, every developer has a **complete local copy of the repository, including the history**. Git is the most well-known example. Even with Git, teams often use a common remote repository (e.g., GitHub), but technically, this remote is not the only complete source.

Advantages

- Many operations are local & very fast
- Working offline is possible
- Every clone is an additional backup
- Powerful branching & merging

Disadvantages

- Conceptually more demanding at the start
- Terms like push, pull, fetch, merge must be clearly understood
- Can become confusing without clear team conventions



Direct Comparison: Centralized vs. Distributed

Aspect	Centralized	Distributed
Project History	Primarily on the central server	Fully available locally
Working without Network	Severely restricted	Easily possible
Speed	Often server-dependent	Frequently very fast locally
Server Outage	Critical bottleneck	Less critical
Typical Systems	SVN, CVS	Git, Mercurial
Branching & Merging	Often more cumbersome	Very strongly supported

❏ **Key Takeaway:** Git is distributed – this shapes almost everything you do with Git.

Benefits of Version Control in Daily Workflow



Undo mistakes

Trace changes cleanly—no need to search through old ZIP files. People work more confidently when changes remain traceable.



Collaboration without overwriting

Version control merges changes, makes conflicts visible, and creates a shared history.



Documentation through project history

When was a feature introduced? When was a bug fixed? Why was a technical decision made? The history provides the answers.



Further Everyday Benefits



Experiment safely

Try out new ideas without jeopardizing the stable state. Version control makes experimentation manageable.



Learn better & think more clearly

Break work down into traceable steps, bundle changes consciously, document decisions, and analyze your own mistakes instead of hiding them.



Not just for source code

Also useful for configuration files, infrastructure definitions, documentation, scripts, and Markdown notes—wherever content evolves.

GIT INSTALLATION

Install & Configure Git

Before you start working with Git, you need two things:

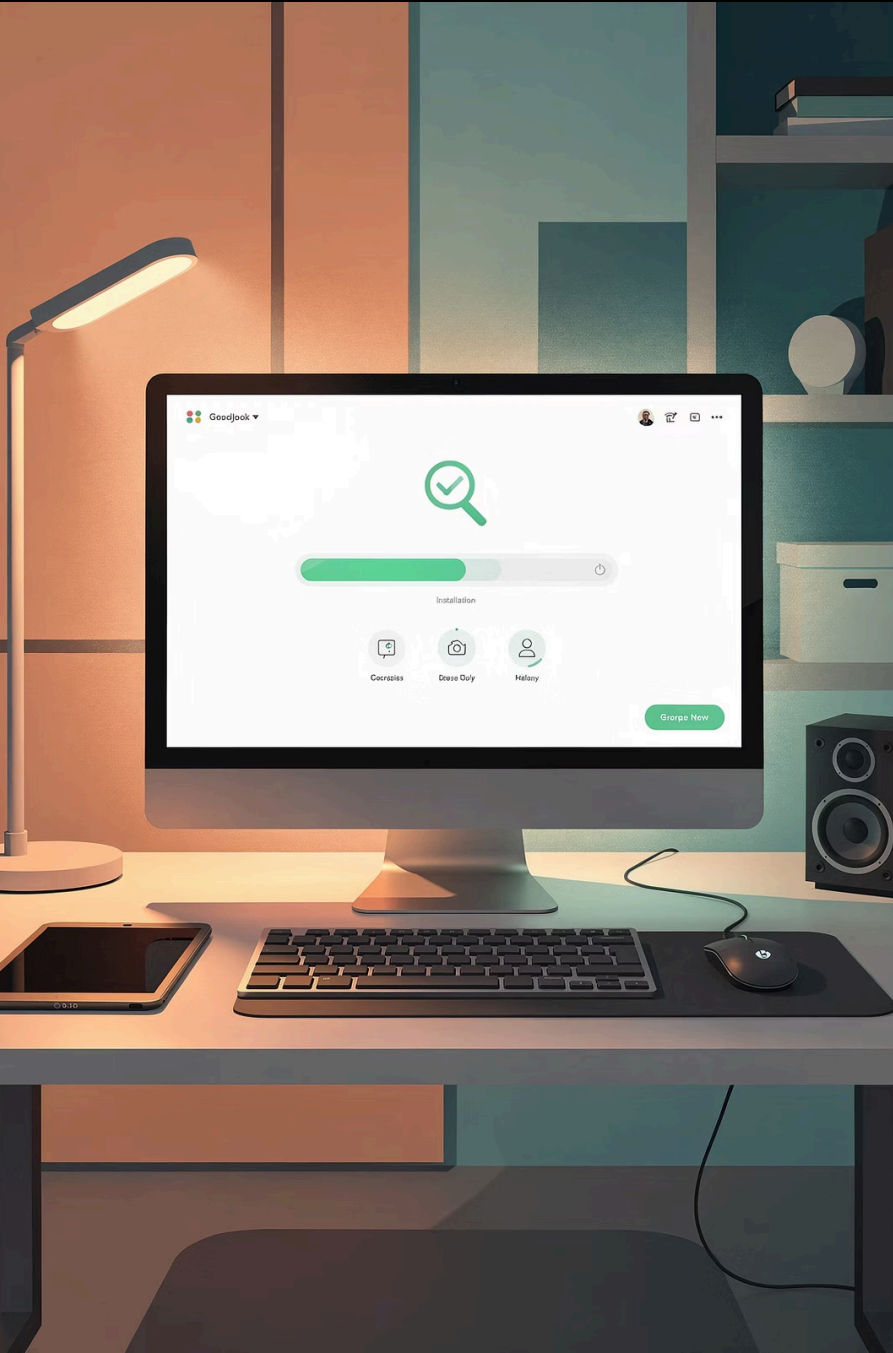
1. Install Git

Git must be installed on your system – the method depends on your operating system.

2. Configure Git

Git should be configured so that your commits correctly reflect your **name and email address**.

The official installation methods are described on the Git website and in the Pro Git book.



Install Git: Windows, macOS & Linux

1

Windows

Official installer from **Git for Windows** via git-scm.com. Keep the default options, then open Git Bash or the terminal.

2

macOS

Via **Homebrew**: `brew install git`, or via Xcode Command Line Tools, or directly from the official Git source.

3

Linux

Via the package manager: `sudo apt install git` (Debian/Ubuntu), `sudo dnf install git` (Fedora), `sudo pacman -S git` (Arch).

For all systems: Verify the installation with `git --version`. A version number confirms a successful installation. If "command not found" appears, it is usually an installation, PATH, or restart issue.

Basic configuration: user.name & user.email

After installation, you should tell Git **who you are**. This information is included as metadata in every commit.

```
git config --global user.name "John Doe"  
git config --global user.email "john@example.com"
```

The three configuration levels

- `--system`: Applies to all users on the computer
- `--global`: Applies to your user (recommended)
- `--local`: Applies only to the current repository

Checking the configuration

Use `git config --global --list` to display all set values. Use `git config --list` to see the values currently active in the current repository.

Important: `user.name` and `user.email` are **commit metadata in Git itself** – not automatically a GitHub login.

- ❏ **Common Misconception:** Many people believe that user.name and user.email are automatically linked to a GitHub or GitLab login. Git is the version control system—GitHub is just a hosting service for Git repositories.

Git vs. GitHub: An Important Distinction

Git

The **version control system** itself. Runs locally on your machine. Manages the project history. Stores commit metadata such as name and email independently of any online platform.

GitHub / GitLab / Bitbucket

Hosting services for Git repositories. These platforms can associate commits with your account if the email address used is known there—but that is a separate function.

GETTING STARTED

Initializing a repository: git init

A **repository** is the place where Git manages the version history of a project. With `git init`, you tell Git that a folder should now be under version control.

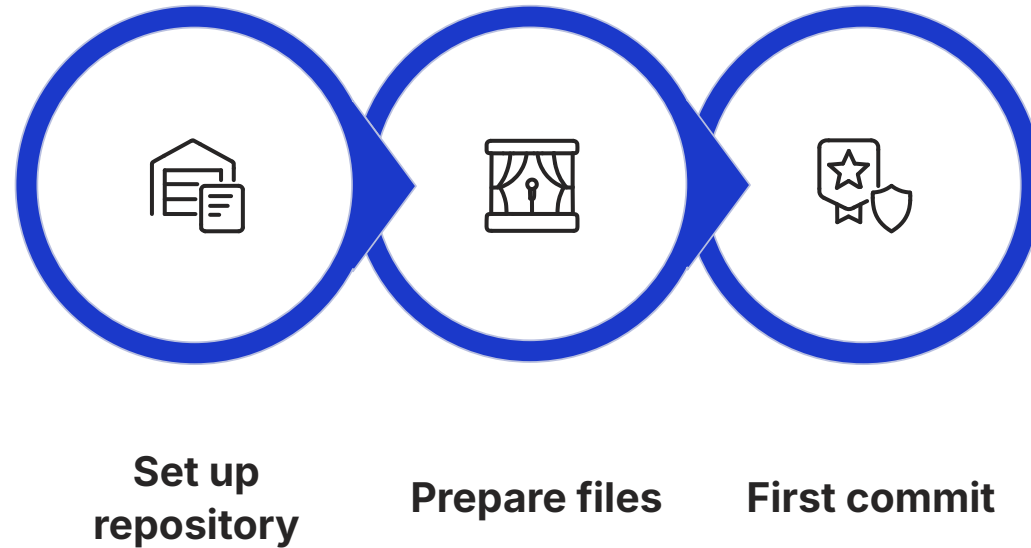
```
mkdir my-project  
cd my-project  
git init
```

Git creates a hidden `.git` folder that contains configuration, branches, the object database, and commit history. Your actual project files remain unchanged.

Important: `git init` alone does not save a version of your work—that requires an initial commit.



What git init does – and what comes next



Existing projects retroactively

You can also run `git init` in a folder where files already exist. Git does not delete or change anything – it just sets up version control.

Are you in a repository?

Use `git status` to check: If you are in a Git repository, Git will show the current state. Outside of one, it will display: "not a git repository".

Git Help: Using Self-Help Effectively

Git comes with excellent built-in help. Working professionally also means using documentation instead of just memorizing commands.

01

General Help

`git help` – general overview. `git help -a` – list of all available subcommands.

03

Quick Reference

`git init -h` – compact command help directly in the terminal. Faster than `--help`.

02

Help for a Command

`git help init` or `git init --help` – opens the complete documentation for the command.

04

Orientation in Daily Workflow

`git status` – the most practical self-help option of all. Shows the state, modified files, branch, and staged changes.

The golden rule of thumb for uncertainty

1

git status

What is the current state of my repository?

2

git diff

What has changed specifically?

3

git log

What happened in the history?

With these three commands, many situations can be understood on your own. If you encounter an unknown command: first use `git help <command>` – pay attention to NAME, SYNOPSIS, DESCRIPTION, and OPTIONS.

📌 **Principle:** If you are unsure – first `git status`, then `git help <command>`, only then execute the command.

The Mental Framework for Getting Started

Repository

The managed project context – `git init` turns a folder into one.

Snapshots & Commits

Git saves project states as traceable snapshots in the history.

Commit Metadata

`user.name` and `user.email` belong to every commit – configure them globally.

Orientation

`git help` and `git status` are essential tools – use them first, always.

This fundamental model is the basis for all further topics: `add`, `commit`, `branch`, `merge`, `clone`, `push`, and `pull` build directly upon it.